Introduction to Algorithms.

- Algorithm is a finite, well defined set of steps for solving a problem.

A good algorithm should:

- be correct
- be composed of a finite number of well-defined steps.
- produce output.
- be able to terminate in finite time.
- be efficient.

Program.

- Is an implementation of an algorithm using a programming language.

Good programming should be.

- able to execute correctly
- able to execute efficiently.
- easy to read and understand.
- easy to maintain.

Extra >>>>>>>>>>

<< stream insertion operation

cout ⇒ character out.

CamScanner

Q: Why does the OS need an exit status?

A: exit status tells the OS whether the program succeeded or failed. Program A ⟶ program B ⟶ program C.

Status 0 ⟶ 1 ⟶ X

Q: Why do we need a namespace.

A: To prevent name collisions. Allows different libraries to use the same names safely.

```
void print();
void print();
```
✗

```
namespace A {
    void print();
}
name space B {
    void print();
}
```

```
A :: print();
B:: print();
```
✓

Q: Which variable need explicit initialization?

A: Local ⟶ must be initialized

Global ⟶ Automatically initialized

Static ⟶ Automatically initialized.

Retains its value between function calls.

Q: What happens if we don't initialize a variable?

A: The value called will be unpredictable.

Q. What is a garbage value?

A: random, leftover data stored in memory before you assign a value to a variable.

Q: What is a garbage collector?

A: collector automatically frees memory that is no longer being used.

Q: pros and Cons of garbage collectors?

A: pros : prevents memory leaks — safer

Easier programming — Better productivity.

cons: slower execution — uses more memory.

Less control — unpredictable pauses.

= = = = = = =

Extra >>> > > > > >> 2016/07/03

- Sqrt is a function in cmath library
- array declaration

  Int arr [] = { }
- Continue.
  - is a loop control statement.
  - skips the rest of the current iteration of a loop and immediately start the next iteration
    - > Makes code cleaner
    - > Improves readability
    - > Reduces nesting

No. Date

break.

- Immidiately terminates loops
  - > Stops unnecessary work
  - > Useful for input validation
  - > Improves efficiency

To break from all loops we can use either return or functions according to the context.

exit (0) is valid syntax but not recommended as it may lead to data leaks.

Pointer arithmatic: Instead of changing the value stored in memory, pointer arithmetic changes the memory address a pointer refers to.

#define: is a preprocessor directive in C++. It tells the preprocessor to perform a text-substitution before the actual compilation begins.

```
#define NAME replacement;
```

Common uses: constants, macros,

Advantages: simple text replacement

No run time overhead

Disadvantages: No type checking

No scope

Harder to debug

Can produce unexpected output.

Makes code less readable.

Ternery operator (called so because there are three operands

condition ? true : false;

Pros: shorter code

better for simple assignments

can be used when a const needs to be initialized according to a condition.

Poimers must initialize at least as a null pointer otherwise unexpected results can occur.

```
int *p = 0;   // null pointer
int n = 5,    // pointer and the pointed types must be same
p = &n        // & = address of operator
cout << *p    // 5   value at operator.
```

If not initialized it becomes wild pointers. They point at garboge addresses.

Need: Modify variables from another function.

Dynamic memory allocation.

```
int *marks = new int[n];
```

Efficiently working with arrays.

No.

Date

Program memory

code } programe lifetime

compiled instructions

Global / static memory

global variables — static variables.

Heap dynamic allocation

Dynamic memory (new/delete) ↑ (grows)

Function lifetime } Stack memory that compiler manages automatically

Function calls local variables. ↓ → fast → automatic → limited size

2026/03/10

How to initate a dynamic variable at the time of declaration.

```
int *p = new int(10);
```

Memory leak: Occurs when a program allocates memory but never realeases it after its no longer needed.

```
delete p;
```

Dynamic arrays: Array whose time is determined at runtime.

```
int arr = new int[n];
delete[] arr
```

sizeof(p) → size of pointer variable depend on the architecture not on the type.

CamScanner

Why do we need a type for a pointer?

Because the compiler needs to know what type of data is stored at the address it points to.

Pointer arithmatics: Performing arithmatic operations on pointers to move between memory locations.

```
int *p
p++      //moves to next integer (+4bytes) not one
p--      byte.
```

char → 1 byte int → 4 bytes double → 8 bytes.

Pointers and arrays.

- The array name represent the address of first element.

arr ⇒ &arr[0]

int *p = arr

p == arr

accessing elements.

arr[0] == *(p)

Passing arrays to functions:

```
int function(int arr[] / int* arr )
    function(arr)
```

value parameters: A copy of the argument is passed.

Reference parameters. create an allias for the orlginal variable.

```
void change(int &x)
change (x)
```

pointer parameters: Instead of the variable address is passed.

(int *p)

(&p)

We need pointers even if we have reference because reference can't access null type. for nullable ~~fy~~ variables we need pointers. : Nothing will happen but an error won't occur.

We need reference even if we have value because:

- When an array is passed a seperate copy is created if value is used. It is slow and uses extra memory.
- Reference are also useful when the function needs to modify the caller's variable.

Default parameters: void greet (string name, string role="guest")

// Hello pipuni:admin greet("pipuni", "admin")

// Hello pipuni:guest. greet ("pipuni")

= = = = = ==

Pseudocode.

- A kind of structured English for specifying algorithms.
- Cannot be compiled nor executed.
- No strict syntax rules.
- Easy to read and understand
- Programming language independent.

Date

ex:

passed by reference

ALGORITHM search(REF list[] INTEGER, size INTEGER,
target INTEGER, REF index INTEGER)
finds target in unordered list ← one line about purpose

Conditions on input ⇒ PRE
list is an array with at least one element
size is number of elements in list.
target contains integer to find.

Conditions on output ⇒ POST
if found, index gives matching index.

Conditions on return values ⇒ RETURN
true if found, otherwise false.

```
1.  index ← 0
2   LOOP (index < size)
3       IF (target = list[index])
4           RETURN true
5       END IF
6       index ← index + 1
7   END LOOP
8   RETURN false
END search
```

• statements are numbered and indented for clarity

Efficiency of an algorithm.

- Amount of memory and time needed to run a program written with a particular algorithm.
- Determining efficiency:
  - Performance measurement → Experimental
  - Performance analysis → Pre determination preferred.

Space complexity.

- Amount of memory that a program with a particular algo needs to run to its completion.

Why is it important?

- To know whether available memory is enough to run the program written with our algorithm.
- Programs that consumes less memory when there are several algorithms for a given problem.
- Could estimate the largest problem that our algo can solve against the memory available.

Components.

1. Instruction space
2. Data space
3. Environmental stack space.

Instruction space.

- Memory needed to load the compiled version of a program.

Doperds on:

1. Compiler used (machine code created from same source code can vary based on compiler).
2. Compiler options used: ex: execution time optimization.
3. Architecture of the target machine.

Data space:

- Memory needed to hold the all the constants and variables of a program.

Environment stack space.

- Memory needed for environment stack.

Environment stack: (runtime stack, call stack).

A data structure that keeps information needed to resume execution of partially completed functions.

Factors of space complexity.

- There are some factors not known at the time of algo. development

Insensitive factors.

+Some factors are insensitive to the problem being solved.

1. Instruction space contribution.
2. Data space contribution from literal constants, named constants and primitive variables.
3. Environment stack space contribution from non-recursive functions.

Sensitive factors.

+Sensitive to the problems being solved.

1. Data space contribution from structured variables & dynamic memory allocation.
2. Environment stack space contribution from recursive functions.

*Space complexity is a function of n (instance characteristic)

Space complexity of algo A:

$$S(n) = c + S_A(n)$$

- Where c is insensitive (constant) space contribution.
- S_A is the sensitive contribution which is a function of instance characteristic n.

Extra >>>>>

Data type	16 bit	32 bit	64 bit
char	1	1	1
bool	1	1	1
short	2	2	2
int	2	4	4
long	4	4	8
float	4	4	4
double	8	8	8
pointer	2	4	8

=====

Time complexity.

- Amount of time that a program written with a particular algorithm needs to run to its completion.

Why are we interested.

- Some problems require a real-time response to their solution.
- Will prefer the program that runs fast when there are several algorithms for a given problem.
- To guarantee the users that the program written with our algorithm can run its completion without any time-out

Factors of ~~space~~ Time complexity.

- Depends on factors that the space complexity depends.

Time complexity

$$T(n) = c + T_A(n)$$

Determining the ~~sp~~ Time complexity.

Two approaches.

1. Operation count
2. Step count.

Operation count approach.

- Identify one or more key operations of the algo.
- Time complexity is estimated by determining number of times these key operations are performed.

CamScanner

Step count approach.

- Estimated by accounting the time spent for all program steps.
- Estimated by determining the number of steps that program takes to complete

Weaknesses of the operation count approach.

- Doesn't measure actual execution time.
- Choosing the key operation may be difficult

Programming instrumentation: the process of modifying a program by adding extra code to collect info about its execution.

Time analysis

Running times.

Bestcase - Time taken when the best possible thing happens

Worst case - Time taken when the worst possible thing happens.

Average case - Time taken for some probability distribution of instance characteristics n.

Functions of time complexity

n is always positive

$T(n) \geq 0$ for all n,

Why are we interested.

- Would like to know whether the algorithm becomes reasonably or incredibly slow when the problem grows.

Linear functions.

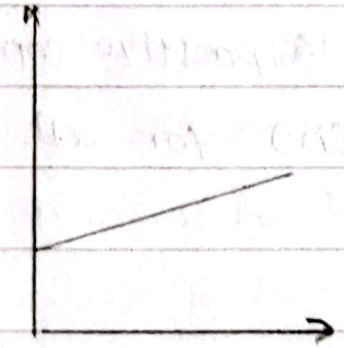
$y = mx + c$ ex: Sequential search.

Logarithmic function.

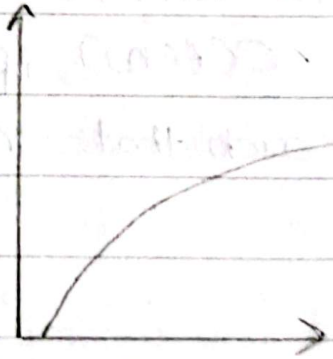
* $xy = m$ ex: binary search.

↳ $\log_x(m) = y$

*binary search is more efficient than sequential search

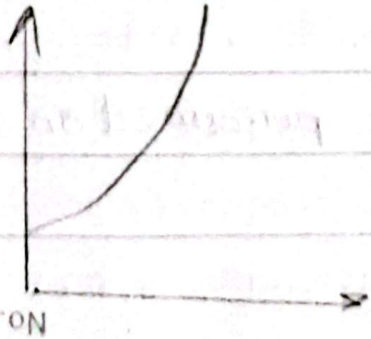


Linear

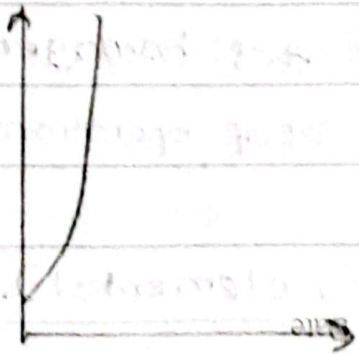


logarithmic.

Linear logarithmic



Quadratic



Asymptotic notation.

Expresses an algorithm's running as its growth rate by dropping the less significant terms and constant coefficient

- This is done to make equations simple.

Growth rate: How fast the function grows with the Input size.

Big Oh notation.

Asymptotic notation for the worst case running time of an algorithm.

Definition: for $T(n) \geq 0$ and $f(n) \geq 0$, $T(n)$ is said to be $O(f(n))$, if there exists positive constant c and n_0 such that $T(n) \leq cf(n)$ for all $n \geq n_0$

Data structures.

Atomic data type: A set of non-decomposable data.

consists of:

A set of homogeneous data.

A set of operations that can be performed on data.

(primitive, elemental).

Composite data type.

A set of decomposable data;
composed using atomic and composite data types.
could consist of a set of
- Homogeneous or
- Heterogeneous data

Data structure.

A structure that holds a set of atomic or composite data
consists of: A set of homogeneous or heterogeneous data.
A set of operations that can be performed on

Stack.

- A linear data structure
- All data additions (push) and removals (pop) are restricted to one end (top).

Last in First out.

Error conditions.

Overflow: Happens when trying to push an item onto a full stack

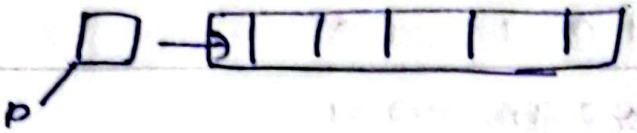
underflow: Happens when attempting to pop an item from an empty stack.

CamScanner

Array based stack.

conceptual physical







Operations

create() destroy() isEmpty() isFull()

pop() push()

an array-based stack requires a predefined maximum capacity, even if the entire capacity is not used.

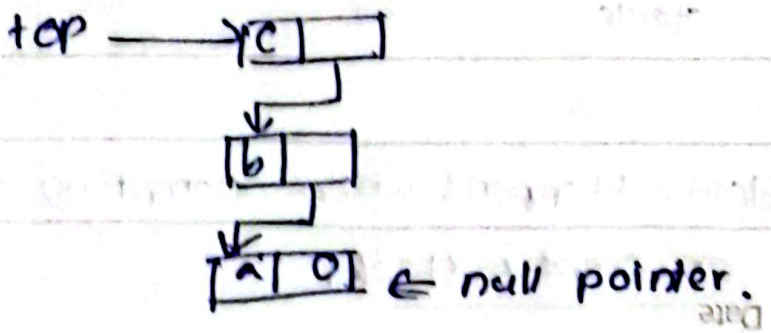
wastes memory in most cases.

can be avoided with linked lists that can grow and shrink dynamically during run time.

Linked list-based stack.

conceptual physical





Operations.

create() destroy() isEmpty()

push() pull()

Stack applications.

Reversing data

converting numbers

parsing data

converting mathematical expressions.

Evaluating mathematical expressions.

Backtracking.

Queues

- A linear data structure.
- Data additions are restricted to a one end and removals can only be done from the other end
- FIFO
- Error conditions
  - Overflow
  - underflow.

operations.

enqueue()

dequeue()

No.

Date

CamScanner

Array based queue.

Global variables.

count = number of elements

front = Index of the first element.

rear = Index of the element after the last one.

size

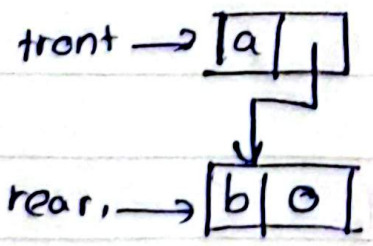
front = rear
when the queue is full or empty.

- create (queueSize)
- destroy()
- isFull()
- isEmpty()
- enqueue (& data)
- dequeue (& data)

rear ← rear MOD size

front ← ~~read~~ front MOD size } to maintain circular nature of an array.

Link list based queues.



CamScanner

osbbal variables.

Type node
    data DATATYPE
    next POINTER to node.
END node.

front POINTER to node, rear POINTER to node.

create()
destroy()
isEmpty()
enqueue(data)
dequeue(& data)

④ p → data = d ≡ (*p). data = d.

Queue applications.

- Process scheduling in OS
- Printer spooling
- Network traffic handling
- Network traffic handling.
- Many other data processing and service providing applications.

No.

Date

CamScanner

Tree

- A set of nodes that holds atomic or composite data
- Nodes are connected to other nodes with edges.
  - > Root
  - > Leaf
  - > Order: Maximum child nodes a node can have.

Binary tree.

- A tree with an order of two.
- A finite set of elements partitioned into three subsets.
  - > Root
  - > Left subtree
  - > Right subtree,

Binary tree traversal.

- Visiting each node of binary tree once and only once.

1. Preorder traversal (each node is visited when it is traversed)
   - > Visit the root
   - > Traverse the left subtree in preorder
   - > Traverse the right subtree in preorder

2. Inorder traversal (each node is not visited till its left subtree is traversed).
   - > Traverse the left subtree in inorder
   - > visit the root
   - > Traverse the right subtree in inorder

CamScanner

3. Postorder traversal (Each node is not visited until both of its subtrees are traversed)

> Traverse the left subtree in postorder
> Traverse the right subtree in postorder
> Visit the root.

Binary search tree.

A binary tree in which

> Data in any node of the left subtree is less than that of the root
> Data in any node of the right subtree is greater than or equal to that of the root.
> Each subtree itself a binary search tree.

When a binary tree is traversed in inorder it becomes a sorting algorithm.

Array based tree.

- Root of the tree is at index 1
- Left child node is 2n
- Right child node is 2n+1

Problem with array based trees.

For a balanced tree of height of q → $2^{h+1}-1$ nodes must be there

∴ 1023.

now assume the tree is highly unbalanced, only 10 nodes will be used. The memory allocated to rest will never be used.

Even still this is a valid option if the tree is balanced and you know the maximum height and also if it doesn't change.

+ If there are duplicated values it is taken as the right child

Tree applications

Represent hierachically related data. (maintaining folders & files)

- ↳ Maintaining a land registry (↑↑ leaf node)
- Represent mathematical expressions.
  - * Expression trees: Every mathematical expression can be devided into 2 operands & operater, root (↓ parent) is operater children are operands.
- Support data compression & encoding
- Efficiently search larger dynamic sets of data.
  - Binary sort can only be done when the array is sorted.

So it is not efficient if we have to insert & delete data, because we have to shift data to maintain the sorted state. Here trees are efficient (Linked list) Because insertions and deletions are efficient in linked list (Just have to change one pointer set). Searching in a linked list is not efficient. So linked list based binary search tree is very efficient for this specific scenario.

CamScanner